



1. 栈的应用

① 括号匹配

括号匹配需要解决的问题：

1. 左括号必须用相同类型的右括号闭合
2. 左括号必须以正确的顺序闭合
3. 每个右括号都有一个对应的相同类型的左括号

每当出现一种左括号，计算机便等待对应的右括号
但在此期间若出现了另一种左括号，则将其视为更高优先级，转而等待新类型的右括号
若遇到当前等待的右括号，则成对消除



1. 栈的应用

②表达式求值

前缀表达式	$+ \ a \ b$
中缀表达式	$a \ + \ b$
后缀表达式	$a \ b \ +$

计算机借助**栈**来实现不同的表达式求值

前缀表达式	+ a b
中缀表达式	a + b
后缀表达式	a b +

1. 栈的应用

②表达式求值

中缀表达式求值	后缀表达式求值	前缀表达式求值
从左到右扫描		从右到左扫描
扫描到操作数 直接入操作数栈		
扫描到操作符 依次弹出栈中优先级 大于等于 当前运算符的所有操作，碰到左括号“(”或栈空停止 每出栈一个操作符，将操作数栈栈顶的两个元素取出进行计算 并将结果压入操作数栈 最后将当前操作符压入操作符栈 表达式扫描结束但操作符栈非空时 依次弹出操作符栈中的符号并计算	扫描到操作符，即将栈顶的两个元素取出进行计算，并将结果压栈	
第一个出栈的操作数放在运算符的右边		第一个出栈的操作数放在运算符的左边
扫描到左括号“(”，入栈 扫描到右括号“)”， 依次弹出栈中运算符并输出 直到左括号“(”弹出为止	前/后缀表达式中没有括号	



1. 栈的应用

③中缀表达式转后缀表达式

$(a + b) * c - d$

按照运算顺序，分别把每个部分

从【操作数1 操作符 操作数2】修改为【操作数1 操作数2 操作符】

$a \ b \ + \ c \ * \ d \ -$



1. 栈的应用

③中缀表达式转后缀表达式

扫描顺序	从左到右扫描
遇到操作数	直接输出，不做栈操作
遇到运算符	依次弹出栈中优先级大于等于当前运算符的所有运算符并输出 碰到左括号“(“或栈空停止 最后入栈当前扫描的运算符
遇到界限符	遇到左括号“(“，入栈 遇到右括号“)“，依次弹出栈中运算符并输出，直到左括号“(“弹出为止



1. 栈的应用

④递归

执行fact (3)

```
int fact(3){
    if(x == 1)
        return 1;
    else return x * 2
}
```

返回最终结果3*2=6

执行fact (2)

```
int fact(2){
    if(x == 1)
        return 1;
    else return x * 1
}
```

返回2*1=2

执行fact (1)

```
int fact(1){
    if(x == 1)
        return 1;
    else return x * fact(x-1);
}
```

返回1

FACT	
X	1
FACT	
X	2
FACT	
X	3

返回1

返回2

返回最终结果6

调用栈



2. 队列的应用

队列由于具有先进先出（FIFO）的特点

所以常用于描述和实现按照进入顺序依次处理的数据对象，例如**排队、缓冲区管理、层次遍历**等问题

为解决计算机主机和打印机之间速度不匹配的问题，通常设置一个打印数据缓冲区，主机将要输出的数据依次写入该缓冲区，而打印机则依次从该缓冲区中取出数据。该缓冲区的逻辑结构应该是

A. 栈

B. 队列

C. 树

D. 图



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

1. 已知程序如下:

```
int S(int n)
{ return (n<=0) ? 0 : S(n-1)+n; }
void main()
{ cout << S(1); }
```

程序运行时使用栈来保存调用过程的信息, 自栈底到栈顶保存的信息依次对应的是

- A. main()->S(1)->S(0)**
- B. S(0)->S(1)->main()
- C. main()->S(0)->S(1)
- D. S(1)->S(0)->main()



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

2. 已知操作符包括“+”“-”“*”“/”“(”和“)”。将中缀表达式 $a+b-a*((c+d)/e-f)+g$ 转换为等价的后缀表达式 $ab+acd+e/f-* -g+$ 时，用栈来存放暂时还不能确定运算次序的操作符。栈初始时空，转换过程中同时保存在栈中的操作符的最大个数是

- A. 5**
- B. 7
- C. 8
- D. 11

祝你今天特别特别开心！